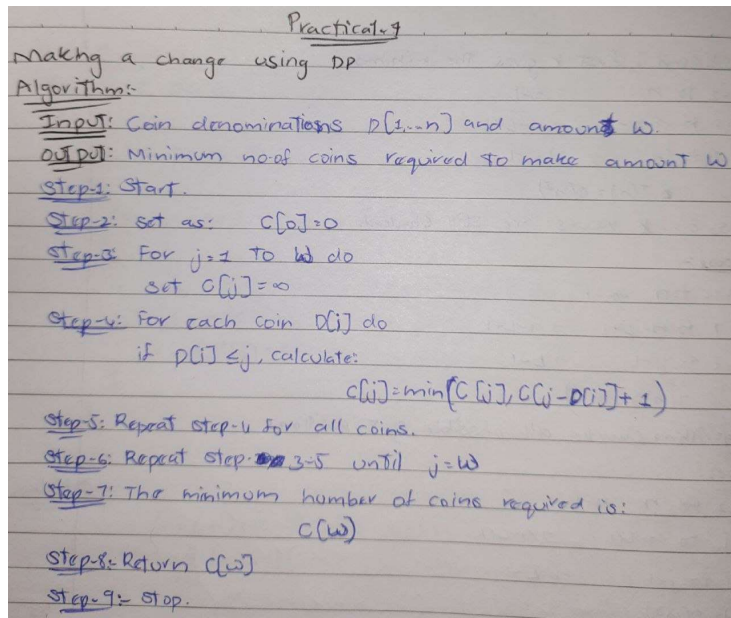


PRACTICAL-07

AIM: Implementation of making a change problem using dynamic programming.

ALGORITHM:



Practical-7
making a change using DP
Algorithm:
Input: Coin denominations $p[1..n]$ and amount w .
Output: Minimum no. of coins required to make amount w .
Step-1: Start.
Step-2: set as: $C[0] = 0$
Step-3: For $j = 1$ TO w do
 set $C[j] = \infty$
Step-4: for each coin $D[i]$ do
 if $p[i] \leq j$, calculate:
 $C[j] = \min(C[j], C[j - p[i]] + 1)$
Step-5: Repeat step-4 for all coins.
Step-6: Repeat step-3-5 until $j = w$
Step-7: The minimum number of coins required is:
 $C(w)$
Step-8: Return $C(w)$
Step-9: Stop.

PROGRAM:

```
#include <iostream>

using namespace std;

int main()
{
    int n, W;

    cout << "Enter number of coins: ";

    cin >> n;

    int D[n];

    cout << "Enter coin denominations: ";

    for (int i = 0; i < n; i++)
        cin >> D[i];

    cout << "Enter amount: ";
```

```
cin >> W;

int C[W + 1];

C[0] = 0;

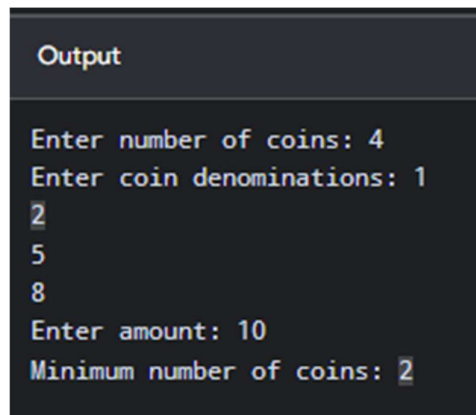
for (int j = 1; j <= W; j++)
{
    C[j] = W + 1;

    for (int i = 0; i < n; i++)
    {
        if (D[i] <= j)
        {
            C[j] = min(C[j], C[j - D[i]] + 1);
        }
    }
}

cout << "Minimum number of coins: " << C[W];

return 0;
}
```

OUTPUT:



```
Output

Enter number of coins: 4
Enter coin denominations: 1
2
5
8
Enter amount: 10
Minimum number of coins: 2
```

TIME COMPLEXITY:

Time complexity:

```
For j=1 to w → w times
{
  For i=1 to n → n times
  {
    if (D[i] ≤ j) { → 1
      c[j] = min(c[i], c[j-D[i]] + 1) → 1
    }
  }
}
```

$T(n, w) = w \times n$
 $T(n, w) = O(nw)$

Best case:
Even if the first coin gives the minimum value the algorithm still checks the remaining coins for that amount. So, all n coins are checked for all w amounts.
 $T(n, w) = w \times n$
 $T(n, w) = O(nw)$

Average-case:
For Average input, some coins may give a better solution earlier, but the algorithm still checks all coin types for every amount.
 $T(n, w) = w \times n$
 $T(n, w) = O(nw)$

Worst case:
Even when the best coin is found at the last coin, all n coins are checked for every amount.
 $T(n, w) = w \times n$
 $T(n, w) = O(nw)$

Best = Average = Worst = $O(nw)$

CONCLUSION:

The Making Change problem using Dynamic Programming finds the minimum number of coins needed for a given amount. It stores previously calculated results to avoid repeated calculations. The time complexity is $O(nW)$ and the space complexity is $O(W)$. Thus, Dynamic Programming provides an efficient solution to the problem.